1 O que este relatório faz

2 Pacotes e funções auxiliares

3 Banco de dados simulado

4 Treino e teste (80/20) e padronização

5 Modelos

6 Resultado no teste (20%)

7 Ambientômica x fenômica: o que cada camada explica

8 Leitura dos resultados

9 Referências

Code

Questão 3 (Prof. Ivan): modelos para ambientômica e fenômica

Cristhian Milbradt Babeski, PPGA/UFSM

09/10/2026

1 O que este relatório faz

Compara modelos de regressão (15 rodam em qualquer computador; XGBoost e deep learning entram automaticamente se os pacotes `xgboost` e `keras` estiverem instalados) em um banco **simulado** com a estrutura dos dados da tese: quatro variáveis de ambiente (camada ambientômica, como as que vêm do NASA POWER) e as bandas R, G, B mais três índices de vegetação (camada fenômica). A resposta é a severidade de doença foliar (%). As referências estão nos comentários do código e na lista final.

Regra de ouro do relatório: todos os modelos são ajustados no mesmo treino (80%) e avaliados no mesmo teste (20%), com RMSE, MAE e R² (CHAI; DRAXLER, 2014; WILLMOTT; MATSUURA, 2005).

2 Pacotes e funções auxiliares

Hide

```
# Todos os pacotes abaixo são instalados com install.packages("nome").
pacotes <- c("glmnet", "mgcv", "e1071", "ranger", "gbm", "MASS", "nnet", "nlme", "arm",
"caret", "openxlsx", "ggplot2")
for (p in pacotes) library(p, character.only = TRUE)
set.seed(123)   # semente única: o resultado se repete em qualquer computador

# Métricas no teste. RMSE pune mais os erros grandes; MAE é o erro médio absoluto (CHA
I; DRAXLER, 2014; WILLMOTT; MATSUURA, 2005).
metricas <- function(obs, pred) c(RMSE = sqrt(mean((obs - pred)^2)),
                                  MAE  = mean(abs(obs - pred)),
                                  R2   = 1 - sum((obs - pred)^2) / sum((obs - mean(ob
s))^2))
resultados <- list()                                                  # guarda uma lin
ha por modelo
guarda <- function(nome, familia, pred) resultados[[nome]] <<- data.frame(Modelo = nom
e, Familia = familia, t(metricas(teste$sev, as.numeric(pred))))
```

3 Banco de dados simulado

Hide

```
n <- 600                                                  # ordem de grandeza dos e
nsaios da tese (250 a 1000 pontos)

# --- Camada ambientômica: covariáveis do tipo NASA POWER (XU, 2016; RESENDE et al., 20
21) ---
temp   <- rnorm(n, 24, 3)                                 # temperatura média (°C)
umid   <- pmin(pmax(rnorm(n, 75, 10), 40), 100)           # umidade relativa (%)
precip <- rgamma(n, shape = 2, scale = 15)                # precipitação (mm)
rad    <- rnorm(n, 18, 3)                                 # radiação (MJ/m2/dia)

# --- Severidade: umidade e chuva favorecem a doença; a temperatura tem um ótimo (respo
sta quadrática) ---
fav <- 0.6 * scale(umid) + 0.4 * scale(precip) - 0.03 * (temp - 24)^2
sev <- pmin(pmax(30 + 12 * as.numeric(fav) + rnorm(n, 0, 5), 0), 100)

# --- Camada fenômica: a doença reduz o verde e aumenta o vermelho; o azul é quase só r
uído ---
R <- 90  + 0.5 * sev + rnorm(n, 0, 8)
G <- 130 - 0.6 * sev + 0.3 * rad + rnorm(n, 0, 8)
B <- 60  + rnorm(n, 0, 10)

# Índices de vegetação RGB (fórmulas usadas nos artigos 3 e 4 da tese). O ExGR fica de
fora aqui: é combinação linear exata de R, G e B e deixaria a matriz singular.
VARI  <- (G - R) / (G + R - B)
GLI   <- (2 * G - R - B) / (2 * G + R + B)
NGRDI <- (G - R) / (G + R)

dados <- data.frame(sev, temp, umid, precip, rad, R, G, B, VARI, GLI, NGRDI)
write.xlsx(dados, "dados_questao3_ivan.xlsx")             # o banco fica salvo em E
xcel
head(round(dados, 3))
```

	sev <dbl>	temp <dbl>	umid <dbl>	precip <dbl>	rad <dbl>	R <dbl>	G <dbl>	B <dbl>	VARI <dbl>
1	32.076	22.319	85.740	35.328	19.414	106.578	133.157	59.115	0.147
2	27.480	23.309	74.727	10.735	20.535	99.913	112.802	59.047	0.084
3	22.998	28.676	74.667	14.413	16.086	111.198	119.178	61.965	0.047
4	20.989	24.212	59.839	35.568	20.702	95.935	122.113	66.210	0.172
5	50.270	24.388	82.904	47.157	20.896	118.430	115.886	53.532	-0.014
6	22.997	29.145	72.893	8.005	11.807	104.157	115.446	67.224	0.074

6 rows | 1-10 of 12 columns

4 Treino e teste (80/20) e padronização

Hide

```
i_teste <- sample(n, 0.2 * n)
treino  <- dados[-i_teste, ]; teste <- dados[i_teste, ]

# Padronizar usando só o treino evita vazamento de informação para o teste (HASTIE; TIB
SHIRANI; FRIEDMAN, 2009).
preditoras <- setdiff(names(dados), "sev")
centro <- sapply(treino[preditoras], mean); escala <- sapply(treino[preditoras], sd)
padroniza <- function(d) as.data.frame(scale(d[preditoras], center = centro, scale = es
cala))
Xtr <- padroniza(treino); Xte <- padroniza(teste)
tr <- cbind(sev = treino$sev, Xtr); te <- cbind(sev = teste$sev, Xte)    # versões padr
onizadas
f_todas <- reformulate(preditoras, "sev")
c(treino = nrow(treino), teste = nrow(teste))
```

```
## treino  teste
##    480    120
```

5 Modelos

5.1 Lineares e paramétricos

Hide

```
# 1. Regressão linear múltipla (mínimos quadrados): Y = b0 + soma(bi Xi) (JAMES et al., 2021).
m_lm <- lm(f_todas, tr);                          guarda("Regressão linear (Lm)", "Linea
r", predict(m_lm, te))

# 2. Stepwise: tira ou coloca variáveis pelo AIC (AKAIKE, 1974).
m_step <- step(m_lm, trace = 0);                  guarda("Stepwise (AIC)", "Linear", pre
dict(m_step, te))

# 3. Regressão com interação entre bandas: R, G, B e produtos entre elas (modelo do pro
jeto, sem citação específica).
m_int <- lm(sev ~ (R + G + B)^3 + temp + umid + precip + rad, tr)
guarda("Regressão com interação R x G x B", "Linear", predict(m_int, te))

# 4. Mínimos quadrados generalizados: aceita variância do erro que cresce com a média
(PINHEIRO; BATES, 2000).
m_gls <- gls(f_todas, data = tr, weights = varPower(form = ~ fitted(.)))
guarda("GLS (variância heterogênea)", "Linear", predict(m_gls, te))
```

5.2 Robusto e bayesiano

Hide

```
# 5. Regressão robusta: M-estimador de Huber, pesa menos os outliers (HUBER, 1964; VENA
BLES; RIPLEY, 2002).
m_rlm <- rlm(f_todas, tr, psi = psi.huber);       guarda("Regressão robusta (rlm, Hube
r)", "Robusto/Bayes", predict(m_rlm, te))

# 6. Regressão bayesiana: prior fraco (Cauchy) sobre os coeficientes (GELMAN et al., 20
08).
m_bayes <- bayesglm(f_todas, data = tr, family = gaussian)
guarda("Regressão bayesiana (bayesglm)", "Robusto/Bayes", predict(m_bayes, te))
```

5.3 Penalizados

Hide

```
xtr <- as.matrix(Xtr); xte <- as.matrix(Xte)
# 7. Ridge: penalidade L2, encolhe coeficientes de preditoras correlacionadas (HOERL; K
ENNARD, 1970).
cv_r <- cv.glmnet(xtr, tr$sev, alpha = 0)         # lambda escolhido por validação cruza
da (FRIEDMAN; HASTIE; TIBSHIRANI, 2010)
guarda("Ridge", "Penalizado", predict(cv_r, xte, s = "lambda.min"))

# 8. Lasso: penalidade L1, zera coeficientes e seleciona variáveis (TIBSHIRANI, 1996).
cv_l <- cv.glmnet(xtr, tr$sev, alpha = 1)
guarda("Lasso", "Penalizado", predict(cv_l, xte, s = "lambda.min"))
coef(cv_l, s = "lambda.min")                       # quais variáveis o Lasso manteve
```

```
## 11 x 1 sparse Matrix of class "dgCMatrix"
##                       s1
## (Intercept) 26.87354200
## temp         0.06395914
## umid         4.76442609
## precip       3.13171315
## rad          0.60714813
## R            2.09409570
## G           -2.89615222
## B           -0.35051176
## VARI         .
## GLI          .
## NGRDI        .
```

5.4 Não lineares e aditivos

Hide

```
# 9. GAM: soma de curvas suaves (splines), uma por variável (HASTIE; TIBSHIRANI, 1986; WOOD, 2017).
m_gam <- gam(sev ~ s(temp) + s(umid) + s(precip) + s(rad) + s(R) + s(G) + s(B) + s(VARI) + s(GLI) + s(NGRDI), data = tr, method = "REML")
guarda("GAM", "Não linear", predict(m_gam, te))

# 10. Regressão não linear (nLnn): forma paramétrica com ótimo térmico, sev = b0 + b1 umid + b2 precip + b3 (temp - topt)^2 + b4 VARI (SEBER; WILD, 1989).
m_nl <- nls(sev ~ b0 + b1 * umid + b2 * precip + b3 * (temp - topt)^2 + b4 * VARI, data = treino,
            start = list(b0 = 30, b1 = 0.5, b2 = 0.1, b3 = -0.3, topt = 24, b4 = -50))
guarda("Regressão não linear (nls)", "Não linear", predict(m_nl, teste))
```

5.5 Vizinhos, margem e árvores

Hide

```
# 11. KNN: média da resposta dos k vizinhos mais próximos (COVER; HART, 1967). k = 7 fixo, por simplicidade.
guarda("KNN (k = 7)", "ML", predict(knnreg(xtr, tr$sev, k = 7), xte))

# 12. SVM com kernel radial: hiperplano de margem máxima (CORTES; VAPNIK, 1995); custo e gamma por validação cruzada.
ajuste_svm <- tune.svm(f_todas, data = tr, cost = 2^(0:4), gamma = 2^(-6:-2))
guarda("SVM (radial)", "ML", predict(ajuste_svm$best.model, te))

# 13. Random Forest: média de 500 árvores sorteadas com reposição (BREIMAN, 2001a; WRIGHT; ZIEGLER, 2017).
m_rf <- ranger(f_todas, data = treino, num.trees = 500, importance = "permutation", seed = 123)
guarda("Random Forest", "ML", predict(m_rf, teste)$predictions)
sort(m_rf$variable.importance, decreasing = TRUE)   # importância por permutação
```

```
##       umid      NGRDI       VARI     precip          G        GLI          R       t
emp          B        rad
## 34.7035655 19.9951599 16.3071045  9.7351666  8.3767743  6.9863979  4.6788959  2.8191
985  1.2716170  0.8926205
```

5.6 Boosting e redes neurais

Hide

```
# 14. Gradient Boosting (GBM): árvores pequenas somadas, cada uma corrigindo o erro das
anteriores (FRIEDMAN, 2001).
m_gbm <- gbm(f_todas, data = treino, distribution = "gaussian", n.trees = 500, shrinkag
e = 0.05,
             interaction.depth = 2, cv.folds = 5, verbose = FALSE)
guarda("Gradient Boosting (GBM)", "ML", predict(m_gbm, teste, n.trees = gbm.perf(m_gbm,
method = "cv", plot.it = FALSE)))

# 15. XGBoost: boosting com regularização das árvores (CHEN; GUESTRIN, 2016). Só roda s
e o pacote estiver instalado.
if (requireNamespace("xgboost", quietly = TRUE)) {
  m_xgb <- xgboost::xgboost(data = xtr, label = tr$sev, nrounds = 200, max_depth = 3, e
ta = 0.05, verbose = 0)
  guarda("XGBoost", "ML", predict(m_xgb, xte))
} else message("xgboost não instalado: rode install.packages('xgboost') para incluir es
te modelo.")

# 16. Rede neural artificial rasa: uma camada oculta com 5 neurônios (RUMELHART; HINTO
N; WILLIAMS, 1986).
m_rna <- nnet(f_todas, data = tr, size = 5, decay = 0.1, linout = TRUE, maxit = 500, tr
ace = FALSE)
guarda("Rede neural (1 camada)", "ML", predict(m_rna, te))

# Aprendizado profundo (várias camadas ocultas; LECUN; BENGIO; HINTON, 2015): exige ker
as/TensorFlow, por isso fica opcional.
if (requireNamespace("keras", quietly = TRUE)) {
  library(keras)
  m_dl <- keras_model_sequential() |> layer_dense(64, "relu", input_shape = ncol(xtr))
|> layer_dense(32, "relu") |> layer_dense(1)
  m_dl |> compile(loss = "mse", optimizer = "adam")
  m_dl |> fit(xtr, tr$sev, epochs = 150, batch_size = 32, validation_split = 0.2, verbo
se = 0)
  guarda("Deep learning (3 camadas)", "ML", predict(m_dl, xte))
} else message("keras não instalado: o bloco de deep learning foi pulado.")
```

6 Resultado no teste (20%)

Hide

```
tabela <- do.call(rbind, resultados); rownames(tabela) <- NULL
tabela <- tabela[order(-tabela$R2), ]
tabela[, c("RMSE", "MAE", "R2")] <- round(tabela[, c("RMSE", "MAE", "R2")], 3)
tabela
```

	Modelo <chr>	Familia <chr>	RM… <dbl>	MAE <dbl>	R2 <dbl>
9	GAM	Não linear	4.514	3.657	0.850
10	Regressão não linear (nls)	Não linear	4.582	3.621	0.845
12	SVM (radial)	ML	4.745	3.811	0.834
14	Gradient Boosting (GBM)	ML	4.989	3.998	0.816
5	Regressão robusta (rlm, Huber)	Robusto/Bayes	5.549	4.577	0.773
8	Lasso	Penalizado	5.614	4.657	0.767
3	Regressão com interação R x G x B	Linear	5.615	4.641	0.767
2	Stepwise (AIC)	Linear	5.619	4.665	0.767
6	Regressão bayesiana (bayesglm)	Robusto/Bayes	5.643	4.674	0.765
1	Regressão linear (Lm)	Linear	5.644	4.675	0.765

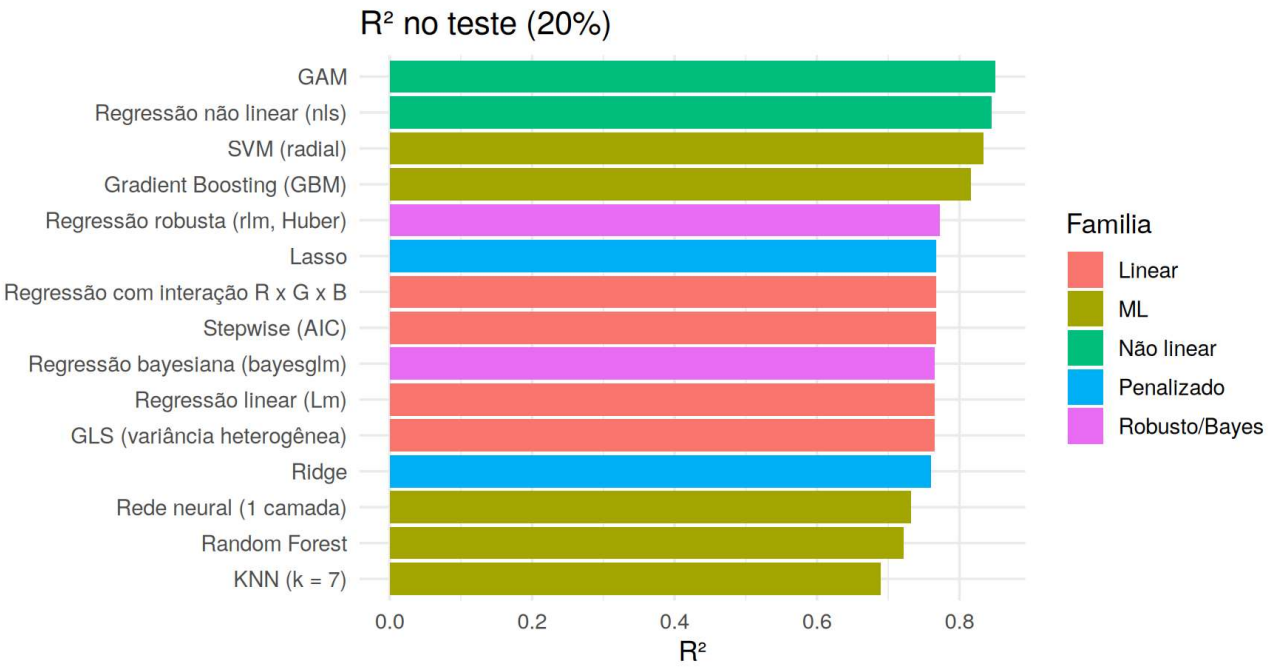
1-10 of 15 rows Previous **1** 2 Next

Hide

```
write.xlsx(tabela, "resultado_modelos_questao3_ivan.xlsx")
melhor <- tabela[which.max(tabela$R2), ]; pior <- tabela[which.min(tabela$R2), ]   # pa
ra o texto abaixo
```

Hide

```
tabela$Modelo <- factor(tabela$Modelo, levels = rev(tabela$Modelo))
ggplot(tabela, aes(Modelo, R2, fill = Familia)) + geom_col() + coord_flip() +
  labs(title = "R² no teste (20%)", x = NULL, y = "R²") + theme_minimal(base_size = 12)
```



7 Ambientômica x fenômica: o que cada camada explica

Mesmo modelo, três conjuntos de preditoras. Mostra quanto da severidade vem do ambiente, quanto vem da imagem e quanto ganha juntando as duas.

Hide

```
amb <- c("temp", "umid", "precip", "rad"); fen <- c("R", "G", "B", "VARI", "GLI", "NGRD
I")
roda <- function(vars, modelo) {
  f <- reformulate(vars, "sev")
  p <- if (modelo == "Linear (Lm)") predict(lm(f, treino), teste) else predict(ranger
(f, treino, num.trees = 500, seed = 123), teste)$predictions
  round(metricas(teste$sev, p), 3)
}
blocos <- expand.grid(Preditoras = c("Só ambiente", "Só imagem RGB", "Ambiente + image
m"), Modelo = c("Linear (Lm)", "Random Forest"))
conj <- list("Só ambiente" = amb, "Só imagem RGB" = fen, "Ambiente + imagem" = c(amb, f
en))
blocos <- cbind(blocos, t(mapply(function(p, m) roda(conj[[as.character(p)]], as.charac
ter(m)), blocos$Preditoras, blocos$Modelo)))
blocos
```

Preditoras <fct>	Modelo <fct>	RMSE <dbl>	MAE <dbl>	R2 <dbl>
Só ambiente	Linear (Lm)	6.723	5.186	0.667
Só imagem RGB	Linear (Lm)	8.273	6.569	0.495
Ambiente + imagem	Linear (Lm)	5.644	4.675	0.765
Só ambiente	Random Forest	6.092	4.761	0.726
Só imagem RGB	Random Forest	8.506	6.702	0.466
Ambiente + imagem	Random Forest	6.144	5.013	0.722

6 rows

8 Leitura dos resultados

O melhor modelo no teste foi **GAM** (R^2 = 0.85, RMSE = 4.514) e o pior **KNN (k = 7)** (R^2 = 0.69, RMSE = 6.482).

Quatro pontos para a discussão:

1. **A diferença veio da forma da relação, não do rótulo “estatística” ou “aprendizado de máquina”.** O GAM e a regressão não linear lideram porque o dado tem um ótimo térmico (curva quadrática) que eles representam direto (HASTIE; TIBSHIRANI, 1986; SEBER; WILD, 1989). Os modelos lineares, penalizados, robusto e bayesiano ficam praticamente empatados, porque com 480 observações todos estimam essencialmente a mesma relação linear; penalização, robustez e prior bayesiano pouco mudam um banco sem outliers.
2. **SVM e boosting vêm logo atrás e entram bem em dados de imagem**, com bandas e índices correlacionados, porque não exigem linearidade nem independência entre preditoras (CORTES;

VAPNIK, 1995; FRIEDMAN, 2001). Random Forest e KNN ficaram atrás neste banco, que é pequeno e quase aditivo (BREIMAN, 2001a; COVER; HART, 1967).

3. **Ambiente e imagem explicam coisas diferentes** (tabela anterior): o ambiente prediz a severidade pela causa (favorabilidade), a imagem pela consequência (verde perdido). No modelo linear, juntar as duas subiu o R^2 de 0.67 (só ambiente) e 0.5 (só imagem) para 0.76. Na Random Forest o ganho não apareceu (0.73 só com ambiente e 0.72 com as duas), o que lembra que mais variáveis não garantem mais acerto.
4. **Prever não é explicar.** Para explicar usa-se o modelo interpretável; para prever em larga escala, o de menor erro no teste (BREIMAN, 2001b).

Limites: dados simulados provam o funcionamento do código, não o desempenho real. Nos dados reais, a mesma rotina precisa de validação cruzada em blocos (parcelas, voos, safras) e de teste independente (ROBERTS et al., 2017).

9 Referências

AKAIKE, H. A new look at the statistical model identification. IEEE Transactions on Automatic Control, v. 19, n. 6, p. 716-723, 1974. BREIMAN, L. Random forests. Machine Learning, v. 45, n. 1, p. 5-32, 2001a. BREIMAN, L. Statistical modeling: the two cultures. Statistical Science, v. 16, n. 3, p. 199-231, 2001b. CHAI, T.; DRAXLER, R. R. Root mean square error (RMSE) or mean absolute error (MAE)? Geoscientific Model Development, v. 7, n. 3, p. 1247-1250, 2014. CHEN, T.; GUESTRIN, C. XGBoost: a scalable tree boosting system. KDD, 22., 2016. p. 785-794. CORTES, C.; VAPNIK, V. Support-vector networks. Machine Learning, v. 20, n. 3, p. 273-297, 1995. COVER, T.; HART, P. Nearest neighbor pattern classification. IEEE Transactions on Information Theory, v. 13, n. 1, p. 21-27, 1967. FRIEDMAN, J. H. Greedy function approximation: a gradient boosting machine. The Annals of Statistics, v. 29, n. 5, p. 1189-1232, 2001. FRIEDMAN, J.; HASTIE, T.; TIBSHIRANI, R. Regularization paths for generalized linear models via coordinate descent. Journal of Statistical Software, v. 33, n. 1, p. 1-22, 2010. GELMAN, A.; JAKULIN, A.; PITTAU, M. G.; SU, Y.-S. A weakly informative default prior distribution for logistic and other regression models. The Annals of Applied Statistics, v. 2, n. 4, p. 1360-1383, 2008. HASTIE, T.; TIBSHIRANI, R. Generalized additive models. Statistical Science, v. 1, n. 3, p. 297-310, 1986. HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. The elements of statistical learning. 2. ed. New York: Springer, 2009. HOERL, A. E.; KENNARD, R. W. Ridge regression: biased estimation for nonorthogonal problems. Technometrics, v. 12, n. 1, p. 55-67, 1970. HUBER, P. J. Robust estimation of a location parameter. The Annals of Mathematical Statistics, v. 35, n. 1, p. 73-101, 1964. JAMES, G.; WITTEN, D.; HASTIE, T.; TIBSHIRANI, R. An introduction to statistical learning: with applications in R. 2. ed. New York: Springer, 2021. LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. Nature, v. 521, p. 436-444, 2015. PINHEIRO, J. C.; BATES, D. M. Mixed-effects models in S and S-PLUS. New York: Springer, 2000. RESENDE, R. T. et al. Enviromics in breeding: applications and perspectives on envirotypic-assisted selection. Theoretical and Applied Genetics, v. 134, p. 95-112, 2021. ROBERTS, D. R. et al. Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. Ecography, v. 40, n. 8, p. 913-929, 2017. RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. Nature, v. 323, p. 533-536, 1986. SEBER, G. A. F.; WILD, C. J. Nonlinear regression. New York: Wiley, 1989. TIBSHIRANI, R. Regression shrinkage and selection via the lasso. Journal of the Royal Statistical Society B, v. 58, n. 1, p. 267-288, 1996. VENABLES, W. N.; RIPLEY, B. D. Modern applied statistics with S. 4. ed. New York: Springer, 2002. WILLMOTT, C. J.; MATSUURA, K. Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance. Climate Research, v. 30, n. 1, p. 79-82, 2005. WOOD, S. N. Generalized additive models: an introduction with R. 2. ed. Boca Raton: CRC Press, 2017. WRIGHT, M. N.; ZIEGLER, A. ranger: a fast implementation of random forests for high dimensional data in C++ and R. Journal of Statistical Software, v. 77, n. 1, p. 1-17, 2017. XU, Y. Envirotyping for deciphering environmental impacts on crop plants. Theoretical and Applied Genetics, v. 129, p. 653-673, 2016.